running the app by hand (as we did at the end of *Saving Your Progress: Pending Specs*) to see for ourselves whether or not it really works. First, boot up the app using the rackup command:

```
$ bundle exec rackup
[2017-06-13 13:34:47] INFO  WEBrick 1.3.1
[2017-06-13 13:34:47] INFO  ruby 2.4.1 (2017-03-22) [x86_64-darwin15]
[2017-06-13 13:34:47] INFO  WEBrick::HTTPServer#start: pid=45899 port=9292
```

With your API server running, try hitting the application's HTTP endpoint from another terminal:

```
$ curl localhost:9292/expenses/2017-06-10 -w "\n"
NameError: uninitialized constant ExpenseTracker::Ledger::DB
    ~/code/expense_tracker/app/ledger.rb:17:in `expenses_on'
    ~/code/expense_tracker/app/api.rb:25:in `block in <class:API>'
```

« *truncated* »

Oh, no—a failure! The DB constant is undefined. When you were first setting up the database, you kept the configuration—including the definition of DB—in config/sequel.rb. You can use a text search tool like grep to find where this file is being loaded in the application.[6] The following command will search the current directory (.) recursively (-r) for the text config/sequel:

```
$ grep config/sequel -r . --exclude-dir=.git
./spec/integration/app/ledger_spec.rb:require_relative '../../../config ↩
/sequel'
```

Right now, the only code that loads this file is the Ledger integration spec. Anything that tries to use the database will only work when this spec file is loaded. That's causing the failure we see when we try to use the app for real. It will also prevent us from successfully running the acceptance specs in spec/acceptance/expense_tracker_spec.rb by themselves.

Being able to run any spec file in isolation is important. You'll often need to do so when you're debugging spec failures, or when you're jumping between writing a class and working with its unit specs.

You can use a little command-line magic to try running each of your spec files individually. Here's an example using bash, the default shell on most Unix-like systems (including the Cygwin and MinGW tools available for Windows):

```
$ (for f in `find spec -iname '*_spec.rb'`; do ↩
    echo "$f:" ↩
    bundle exec rspec $f -fp || exit 1 ↩
  done)
```

6. https://en.wikipedia.org/wiki/Grep